

Modelado UML del proyecto SILVA

Camilo Conde, Miguel Jácome y Juan Rosales

Programa de Ingeniería de Sistemas, Universidad Tecnológica de Bolívar

Ingeniería de Software, grupo 1206

Prof. Aisner José Marrugo Juliao

8 de septiembre de 2026

Modelado UML del proyecto SILVA

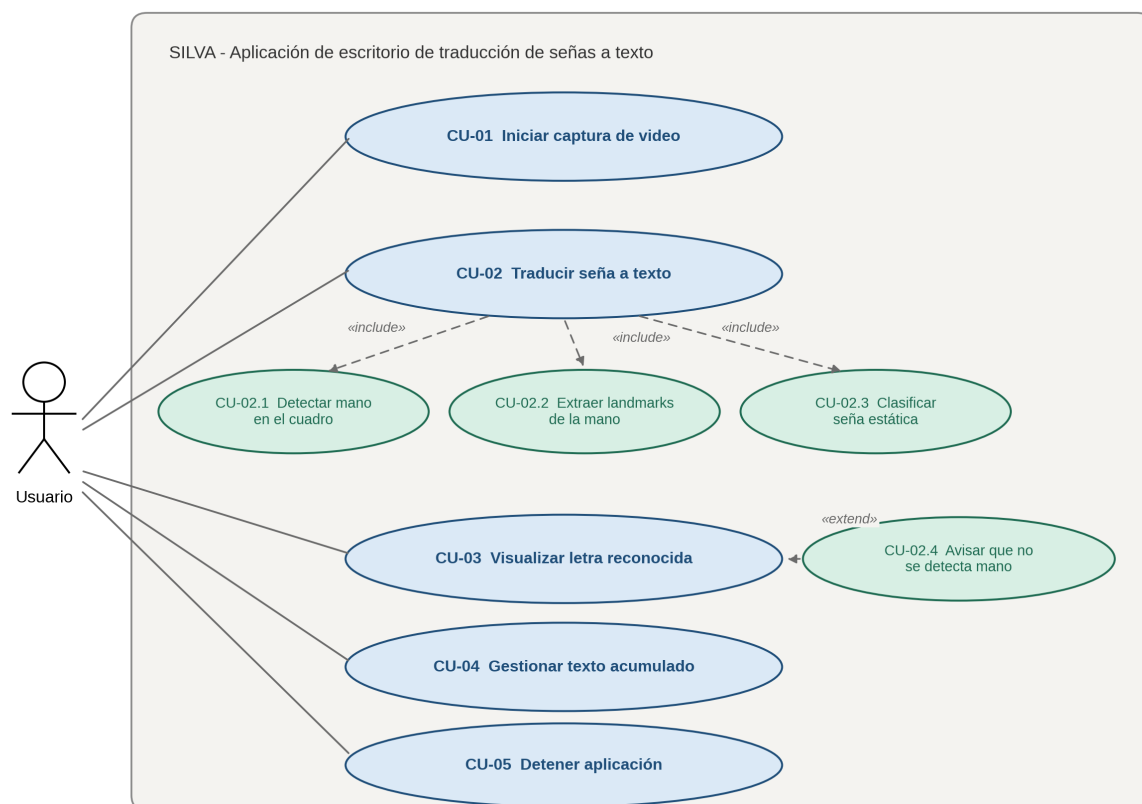
SILVA es una aplicación de escritorio que traduce a texto, en tiempo real, señas estáticas del alfabeto dactilológico capturadas con una cámara web. Los seis diagramas que siguen representan ese mismo sistema en notación UML 2.5.1 (Object Management Group [OMG], 2017): los actores, clases, componentes y nodos que aparecen en cada figura corresponden entre sí y con las funcionalidades definidas para el producto mínimo viable.

Diagrama de casos de uso

El sistema tiene un único actor, el Usuario, que puede ser una persona sorda que desea expresarse por escrito o una persona oyente que necesita interpretar la seña; ambos ejecutan las mismas funciones, por lo que no se justifica separarlos. El actor inicia la captura (CU-01), traduce la seña (CU-02), visualiza la letra reconocida (CU-03), gestiona el texto acumulado (CU-04) y detiene la aplicación (CU-05). El caso de uso central se descompone mediante relaciones de inclusión en las tres tareas que siempre se ejecutan: detectar la mano, extraer los landmarks y clasificar la seña. El aviso de que no hay mano en el cuadro se modela como extensión, porque solo ocurre bajo esa condición. El equipo no entrena un modelo propio, sino que consume uno preentrenado, y por eso no existe un actor entrenador ni casos de uso de recolección de datos.

Figura 1

Diagrama de casos de uso del sistema SILVA



Nota. Elaboración propia. Los casos en verde no son invocados directamente por el actor: se ejecutan por inclusión o extensión desde CU-02 y CU-03.

Diagrama de clases

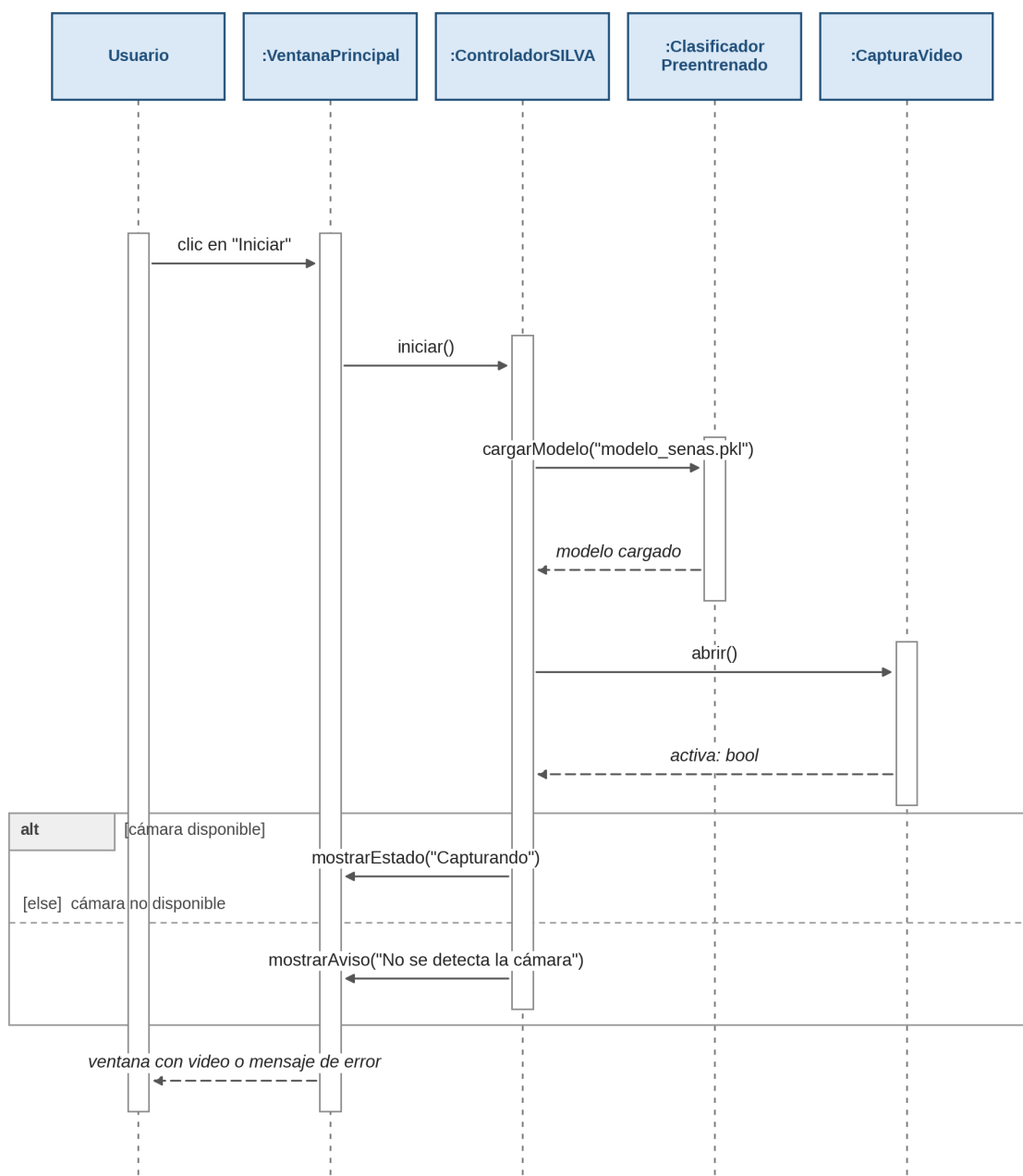
La estructura se organizó por responsabilidades y no a partir de tablas de una base de datos. `VentanaPrincipal` concentra la presentación y los eventos de teclado; `ControladorSILVA` orquesta el ciclo de procesamiento y es el único que conoce a todos los módulos; `CapturaVideo`, `DetectorManos` y `ClasificadorPreentrenado` encapsulan cada uno una biblioteca externa; `BufferTexto` mantiene el texto acumulado y `RepositorioHistorial` aísla la persistencia. Las composiciones indican que los módulos no existen fuera del controlador que los crea y que los veintiún landmarks pertenecen a la mano que los contiene. La asociación con el clasificador apunta a la interfaz `Clasificador` y no a su implementación, de modo que cambiar el modelo preentrenado no obliga a modificar el resto del sistema.

Iniciar captura de video (CU-01)

El arranque carga primero el modelo y después abre la cámara. El fragmento alternativo recoge el caso en que la cámara no está disponible, que es la falla más probable en un equipo de escritorio.

Figura 3

Diagrama de secuencia del caso de uso CU-01, iniciar captura de video



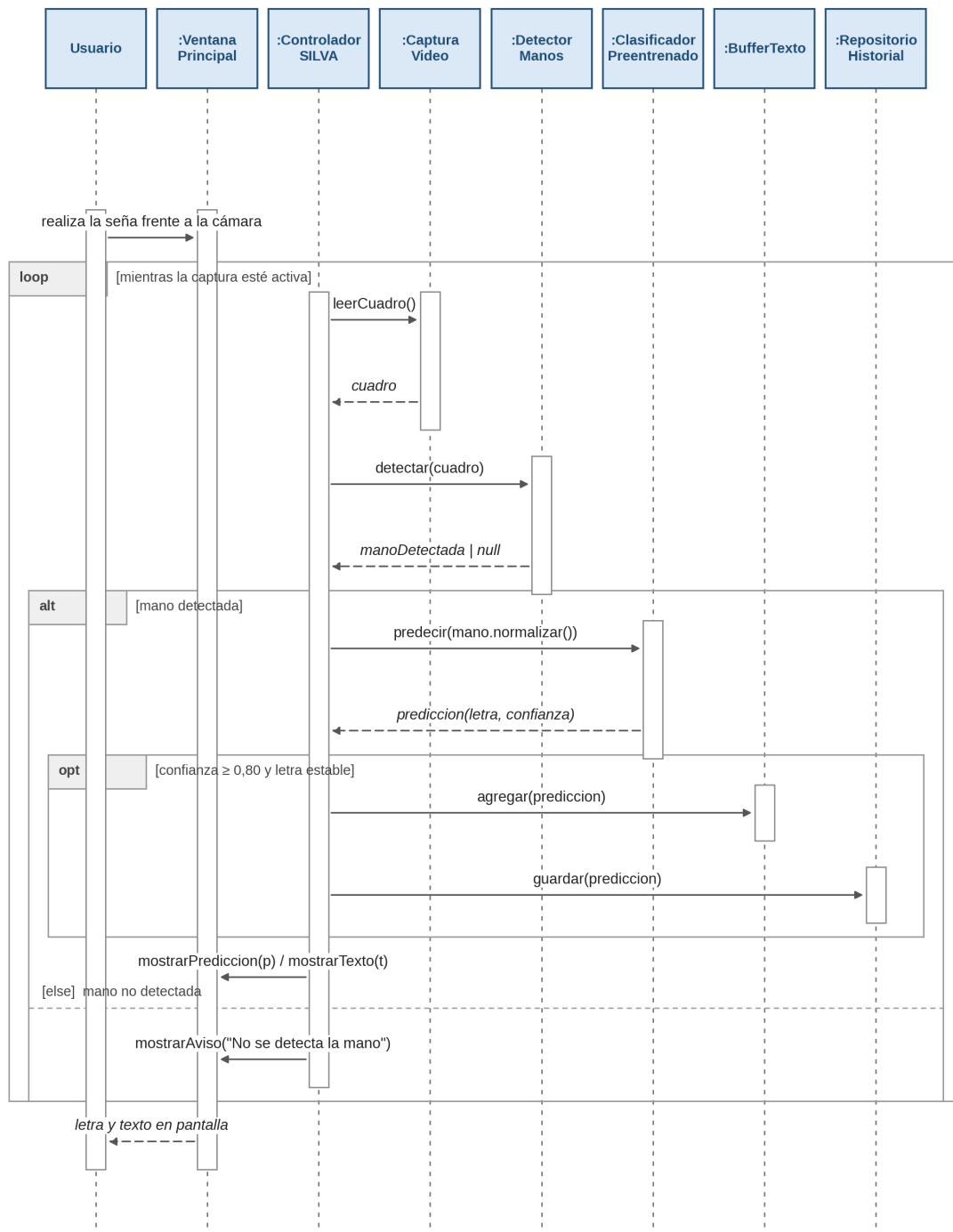
Nota. Elaboración propia. Las flechas continuas representan mensajes síncronos y las discontinuas, mensajes de retorno.

Traducir la seña y mostrar la letra (CU-02 y CU-03)

Este es el proceso central del producto. El fragmento de repetición cubre el ciclo por cuadro; el alternativo separa el caso con mano detectada del que no la tiene, que dispara la extensión CU-02.4; y el fragmento opcional contiene las dos condiciones que debe cumplir una predicción para aceptarse: confianza mínima de 0,80 y la misma letra durante cinco cuadros consecutivos. El registro en el historial ocurre solo cuando la letra se confirma.

Figura 4

Diagrama de secuencia de los casos de uso CU-02 y CU-03, traducir la seña y mostrar la letra



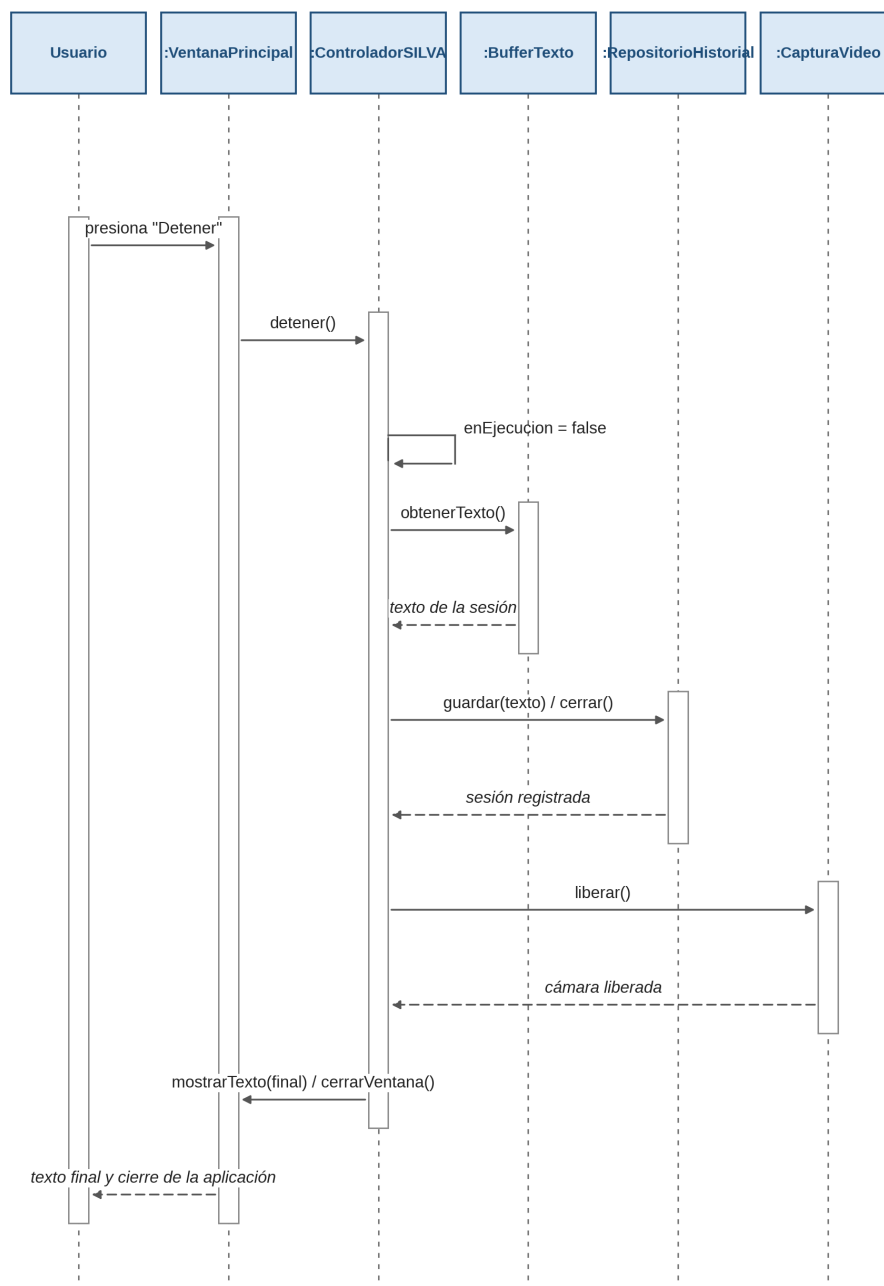
Nota. Elaboración propia. Los fragmentos combinados loop, alt y opt siguen la notación de UML 2.5.1 (OMG, 2017).

Detener la aplicación (CU-05)

El cierre guarda el texto de la sesión, cierra el historial y libera la cámara. Modelar esta secuencia mostró que el orden importa: liberar la cámara antes de guardar dejaría el registro incompleto si el usuario cierra la ventana de inmediato.

Figura 5

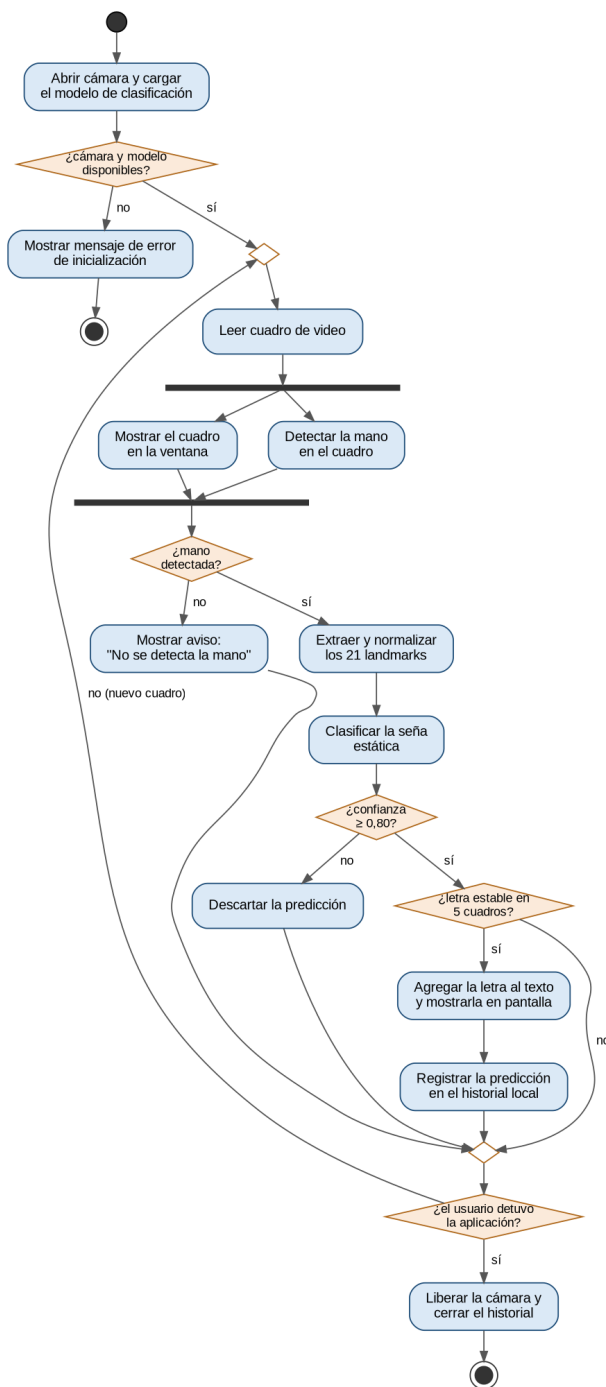
Diagrama de secuencia del caso de uso CU-05, detener la aplicación



Nota. Elaboración propia. El mensaje reflexivo sobre ControladorSILVA representa el cambio de estado que interrumpe el ciclo de captura.

Diagrama de actividades

Se seleccionó el proceso completo de traducción en tiempo real, desde que la aplicación inicia hasta que el usuario la detiene, por ser el flujo que integra a todos los módulos. Con cada cuadro ocurre una bifurcación en paralelo: mientras una rama lo dibuja en la ventana, la otra ejecuta la detección de la mano, y ambas se sincronizan antes de evaluar el resultado. Esa separación es la que evita que el video se congele mientras se procesa la predicción. Las decisiones representan la disponibilidad de recursos, la presencia de mano, el umbral de confianza y la estabilidad de la letra; todos los caminos alternativos convergen en la verificación de si el usuario detuvo la aplicación.

Figura 6*Diagrama de actividades del proceso de traducción en tiempo real*

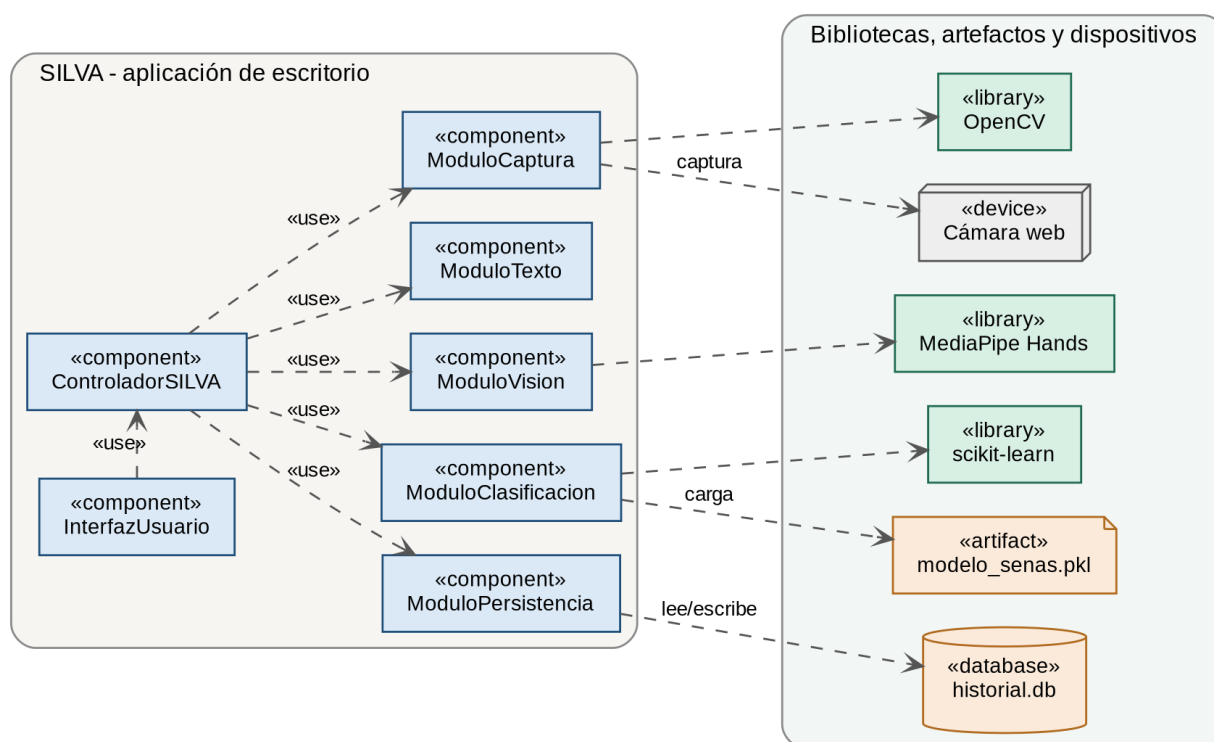
Nota. Elaboración propia. Las barras negras horizontales son los nodos de bifurcación y sincronización de flujos concurrentes; los rombos vacíos son nodos de unión (merge) donde confluyen caminos alternativos antes de una acción o decisión.

Diagrama de componentes

La solución se organiza en siete componentes internos. InterfazUsuario depende únicamente de ControladorSILVA, y este de los cinco módulos funcionales, que a su vez dependen de las bibliotecas y artefactos que consumen: OpenCV para la captura y el despliegue del video (Bradski, 2000), MediaPipe Hands para la detección de la mano y la extracción de landmarks (Lugaresi et al., 2019) y scikit-learn para cargar y ejecutar el modelo preentrenado (Pedregosa et al., 2011). Las dependencias apuntan siempre hacia afuera, de manera que ningún elemento externo conoce al controlador.

Figura 7

Diagrama de componentes de la solución SILVA



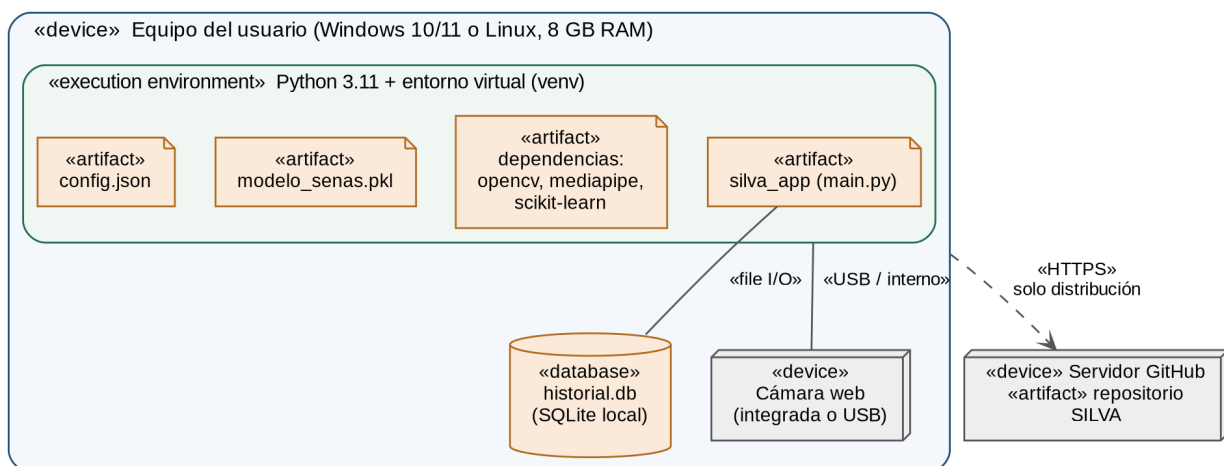
Nota. Elaboración propia. Las flechas discontinuas indican dependencias de uso entre componentes, bibliotecas, artefactos y dispositivos.

Diagrama de despliegue

SILVA se ejecuta por completo en el equipo del usuario. No hay servidor de aplicación, base de datos remota ni servicios en la nube, y tampoco se requiere conexión a internet durante la ejecución. Dentro del equipo, el entorno virtual de Python aloja el ejecutable, las dependencias, el modelo preentrenado y el archivo de configuración; el historial es un archivo SQLite local y la cámara web se conecta como dispositivo integrado o por USB. El servidor de GitHub aparece porque allí reside el repositorio desde el que se distribuye el código, pero se representa fuera del equipo y con una dependencia que no participa en la ejecución.

Figura 8

Diagrama de despliegue propuesto para la versión final de SILVA



Nota. Elaboración propia. Los nodos anidados representan el entorno de ejecución dentro del dispositivo cliente; las líneas continuas son vías de comunicación y la discontinua, una dependencia de distribución.

Coherencia entre los diagramas

Los seis diagramas describen el mismo diseño. Cada caso de uso de la Figura 1 tiene su correspondencia en las figuras siguientes: CU-01 se detalla en la Figura 3, CU-02 y CU-03 en la Figura 4, y CU-05 en la Figura 5. Las líneas de vida de esos diagramas son instancias de las clases de la Figura 2, esas clases se agrupan en los módulos de la Figura 7, y todos esos módulos se ejecutan en el nodo Equipo del usuario de la Figura 8. La Figura 6 recorre el mismo flujo de la Figura 4, pero desde el punto de vista del proceso.

El ejercicio también corrigió tres puntos del diseño inicial. Primero, se eliminaron el actor y los casos de uso asociados al entrenamiento del modelo, que ya no corresponden al alcance. Segundo, aparecieron las condiciones de confianza y estabilidad, que no existían en la descripción original y resultaron necesarias al intentar representar el ciclo por cuadro. Tercero, se hizo explícito el orden de las operaciones de cierre, que solo se evidenció al construir el diagrama de secuencia correspondiente.

Referencias

- Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*, 25(11), 120-125.
- Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.-L., Yong, M. G., Lee, J., Chang, W.-T., Hua, W., Georg, M., y Grundmann, M. (2019). *MediaPipe: A framework for building perception pipelines*. arXiv.
<https://arxiv.org/abs/1906.08172>
- Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), version 2.5.1*. <https://www.omg.org/spec/UML/2.5.1/>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., y Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.